American International University-Bangladesh (AIUB)

Department of Computer Science

Faculty of Science & Technology (FST)

Computer Graphics [A]

Final-Term Project Report

PROJECT TITLE:

2D City Disaster Simulation

Semester:

Summer 2025-26

Supervised by:

Zishan Ahmed Onik

Group: 08		***Section: A***	
SL	**Student Name**	**Student ID**	***Contribution***
1	MD. MURAD HASAN	23-55559-3	
2	MD. RABBY SARKER RONY	24-55981-1	
3	MD. SADMAN AMIN BHUIYAN SUNY		
4	FAHMIDA ISLAM BAYAN	24-58115-2	
5	NOSHIN SUNZIDA ALAM	24-56486-1	

Table of Contents

1. Introduction

2D City Disaster Simulation is a real-time C++ program built with OpenGL and the freeglut toolkit. It draws a small city on the screen using nothing but basic OpenGL primitives - points, lines, triangles and polygons - so the whole scene is generated from code rather than loaded from image files or 3D models. The buildings, road, vehicles, people, trees, weather and rescue vehicles are each drawn by hand with fixed coordinates inside a 1000 x 650 world window.

The application is scene based. The user can move between six different situations - Normal City, Earthquake, Fire, Flood, Storm and Road Accident - at any time with the A and D keys, and each scene changes what appears on the screen and how the objects move. Beyond the six scenes, the project also includes a night mode and an evening mode for the normal city, an intro screen that lists the controls, a live HUD that shows the current scene and status, and a closing screen with the team details.

2. Objectives

• Build a single 2D city that can be drawn and animated in real time using OpenGL and GLUT.

• Show six different emergency situations around the same city, each with a distinct visual effect, moving objects, and status message on the HUD.

• Make the simulation interactive through the keyboard, so the scene, time of day, and fire-hose direction can all be controlled while it is running.

3. Tools & Technologies Used

- Programming Language: C++
- Graphics API: OpenGL and GLU (the GLU helper functions are used only for gluOrtho2D)
- Windowing and Input Toolkit: freeglut (a modern replacement for the older GLUT)
- Text and Fonts: GLUT bitmap fonts (used for the HUD, intro and end screen)
- IDE / Build Environment: Code::Blocks
- Platform: Windows
- Version Control: Git and GitHub

4. Features Implemented

Six Interactive Scenes

- Normal City, Earthquake, Fire, Flood, Storm, and Road Accident — switchable at any time with A / D.

Lighting Modes

- Night Mode: dark sky, stars, a glowing moon, and lit-up building windows.
- Evening Mode: sunset gradient, a setting sun with glow, and long shadows cast by the buildings.

Disaster-Specific Effects

- Earthquake: shaking, tilting buildings and a cracked road.
- Fire: flickering flames, rising smoke, and a user-aimed water jet.
- Flood: a rising, wave-animated water level with a rescue boat.
- Storm: flashing lightning and wind-blown debris.
- Accident: crumpled cars, skid marks, and broken glass.

Background Animation

- Continuously moving clouds, flying birds, falling rain, and a moving sun or moon.

Interactivity

- Full keyboard control: scene switching, pause/resume, reset, night/evening toggle, zoom, and fire-hose aiming (W, J, L, I, K).

Interface

- An intro screen, a closing screen with team credits, and a live HUD that displays the current scene name, a status message, and the available key controls.

5. Algorithms & Mathematical Techniques Used

Two scan-conversion algorithms from the course were implemented manually rather than relying on OpenGL's built-in drawing calls:

5.1 DDA Line Drawing Algorithm

Used to draw the tyre skid marks in the Road Accident scene. Given two endpoints, it steps along the larger of Δx or Δy, incrementing both x and y by equal fractional steps each iteration.

```
int dx = x2 - x1, dy = y2 - y1;

int steps;

if (abs(dx) > abs(dy)) steps = abs(dx);

else steps = abs(dy);

float xInc = dx / (float)steps;

float yInc = dy / (float)steps;

float x = x1, y = y1;

for (int i = 0; i <= steps; i++) {

    plot(round(x), round(y));

    x += xInc;

    y += yInc;

}
```

5.2 Midpoint Circle Drawing Algorithm

Used to draw the rim outline on every car wheel. It plots one octant of the circle using a decision parameter, then mirrors each point across all eight symmetric positions.

```
int x = 0, y = r, d = 1 - r;
plotEightPoints(cx, cy, x, y);
while (x < y) {
    if (d < 0) d += 2*x + 3;
    else { d += 2*(x - y) + 5; y--; }
    x++;
    plotEightPoints(cx, cy, x, y);
}
```

5.3 Simple Motion Patterns Used in the Animation

Apart from the two taught scan-conversion algorithms, most of the motion in the project is built from a small set of simple patterns that are reused across several scenes:

• Flood wave: `y = floodLevel + 7 sin(0.035 x + 0.12 frameCount)` - the edge of the flood water is drawn as a sequence of short line segments, and adding the sine term to each one makes the surface ripple instead of staying flat.

• Character movement: `position = start + (end - start) t`, where `t` runs from `0` to `1` - linear interpolation, used to slide escaping residents and rescued people from a doorway down to a target point on the street.

• Shake and flicker: `offset = A sin(k frameCount)` - the same oscillation with a different amplitude `A` and speed `k` drives the earthquake camera shake, the flame flicker, waving and walking limbs, swaying trees, and bird wing flaps.

• Circles and rays: `x = cx + r cos(theta), y = cy + r sin(theta)` - points spread around a ring at a fixed radius, used for the sun, the layered glow around the moon, and the round lamps.

None of these is complicated on its own; they are used mostly to avoid hard-coding every frame of the animation. The busier scenes, such as the flood, simply combine several of them at once.

6. Instructions section

The controls used in the program are listed below. The N and E keys only affect the normal city scene; the fire-hose keys only have an effect while the fire scene is running.

KEY	WHAT IT DOES
A / D	move to the previous / next scene
P	pause the animation
R	reset the current scene (and the ambulance in the accident scene)
N	turn night mode on (normal city only)
E	turn evening mode on (normal city only)
+ / -	zoom in / zoom out
W	spray water from the fire hose (fire scene)
J / L	aim the hose to the left / right (fire scene)
I / K	aim the hose up / down (fire scene)
X	exit the program

7. Scenario Descriptions

A short walkthrough of each screen, in the order the user sees them. Screenshots are placed alongside each description below.

7.1 Intro Screen

The program starts on a dark title screen showing the project name, the list of all six scenes, and the full control map. Nothing is drawn behind it yet - the city only appears after the user presses a key. The HUD and the scene itself are hidden until then.

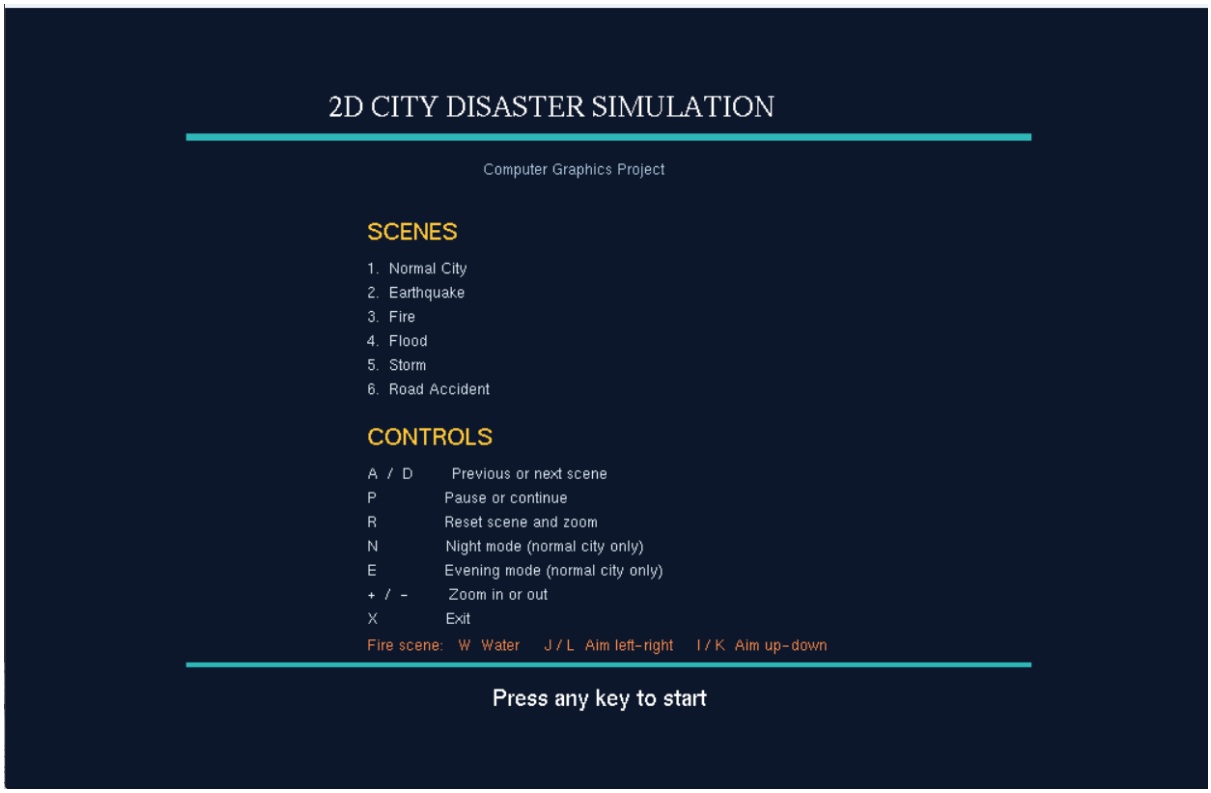

Figure 1: Intro screen - press any key to start the simulation

7.2 Normal City

This is the base scene. The hospital, the tower and the apartment are drawn side by side with trees and street lamps in front of them, a car passes on the road, and birds and clouds move across the sky. From here the user can press N for night mode or E for evening mode. Night replaces the sky with stars and a glowing moon and lights every window warm yellow; evening shifts the whole sky toward orange and purple, drops the sun low between the buildings and draws long shadows across the sidewalk.

Figure 2: Normal city with a car on the road (night and evening modes are also available here)

Figure 2.1: Normal city scene with night and evening variation

7.3 Earthquake

The scene shakes as soon as it is entered, so the buildings visibly lean left and right while the whole frame trembles a little. A dark crack branches across the road, two grey boxes drop from the buildings as debris, and residents run out of the three main doors one after another, pausing to wave before they reach the open ground. The doors swing fully open first; the people only start moving once their own delay has passed.

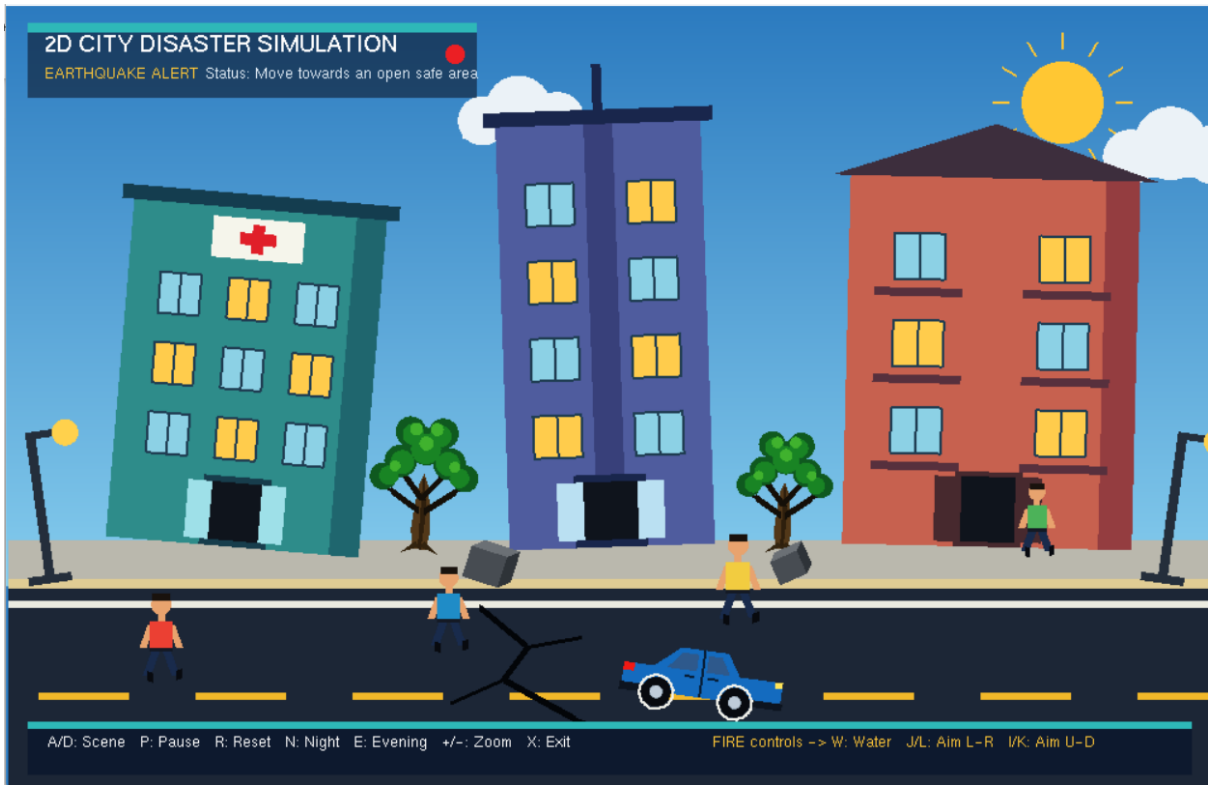

Figure 3: Earthquake - road cracks, falling debris and people running out of the buildings

7.4 Fire

Flames and a column of smoke rise from two windows of the apartment. The fire truck stands on the road with a ladder on top and its siren flashing, and a resident leans out of an open window waving for help. The player aims the hose with J and L, raises or lowers it with I and K, and presses W to spray water. Three people are carried out and walk toward the ambulance, which waits on the right side of the road.

Figure 4: Fire - flames and smoke from the apartment, fire truck and the water jet

7.5 Flood

The water level climbs steadily from the bottom until it covers most of the road. The top edge of the water ripples, small wave lines run across the surface, and a rescue boat crosses from left to right carrying two responders. As the level rises, windows on all three buildings slide open in sequence and a resident appears in each one calling for help. A piece of wood floats and bobs on the surface near the centre.

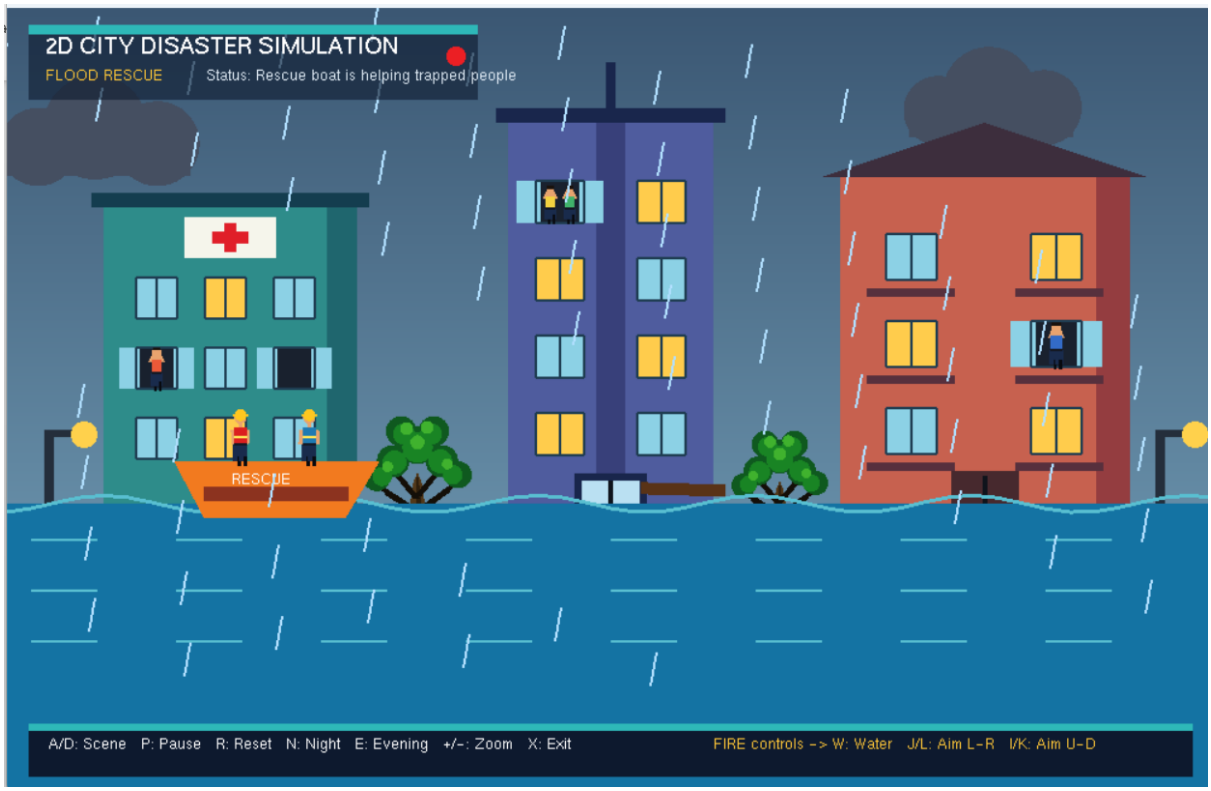

Figure 5: Flood - rising water, the rescue boat and opened windows

7.6 Storm

The sky darkens and lightning flashes in short bursts every few seconds, each flash drawn as a jagged bolt with a fork. Rain falls at an angle and two pieces of debris - a sheet of paper and a leaf - blow across the screen, tumbling as they go. A person walks on the pavement trying to hold an umbrella against the wind.

Figure 6: Storm - lightning, heavy rain and wind-blown debris

7.7 Road Accident

Two cars have collided in the middle of the road and are drawn crumpled, one facing right and one flipped and facing left. Behind them, dark skid marks show where the driver braked, and a scatter of pale glass lies between the two cars. An ambulance drives in from the right and stops, one officer stands in the road directing traffic, and a small crowd and a lying injured pedestrian are gathered on the sidewalk. Cones and barriers close off the two ends of the road so the area is clearly blocked.

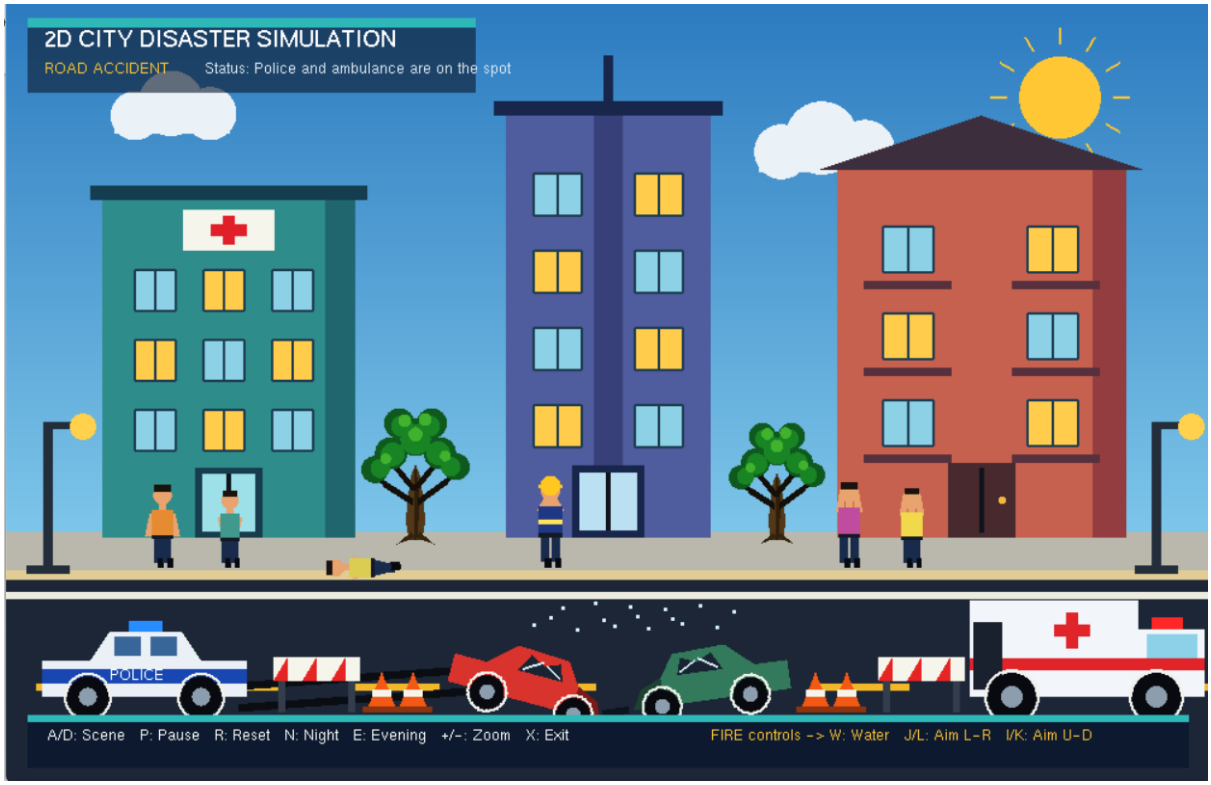

Figure 7: Road accident - the two crashed cars, police, ambulance and the barriers

7.8 Ending Screen

The closing screen appears after the accident scene. It confirms that all six scenes have been shown, lists the five team members, and gives the last three keys: A to go back to the accident scene, R to restart from the intro, and X to quit the program.

Figure 8: Ending screen - completion message, team names and the closing controls

8. Limitations & Future Improvements

The project is a course submission, so it favours simple, predictable behaviour over systems a game engine would provide. The main things that are missing or simplified are listed below.

Limitations:

• Movement is scripted rather than simulated - vehicles, the boat and the people all follow fixed, pre-set paths, so there is no collision detection, no physics and no pathfinding. The scene is also switched by the keyboard instead of being driven by an event, and every run is identical because nothing is random.

• Sound is the other large gap. All the feedback in the project is visual, so the alarm and the weather are silent. For a group project this was a simple and safe choice, but it is also the first thing that would be added in a later version.

Future improvements:

• Real collision detection and simple physics, so the vehicles and the boat stop instead of passing through objects.

• Sound effects per scene (siren, collapsing, rain) using a small audio library.

• A free-moving camera and adjustable disaster intensity, so the same scene plays out a little differently each time.

• Buttons on the screen as an alternative to the keyboard, which would make the demo easier to control on a projector.

9. Conclusion

Looking back, the project successfully transformed a set of simple geometric shapes into a small city that changes through six different situations, from normal daily life to earthquake, fire, flood, storm, and road accident. Because the entire environment is drawn with basic OpenGL primitives, the city remains visually consistent across all scenes, which helps each situation feel like part of the same world rather than separate demonstrations.

The most important part of the work was the animation design—deciding what moves in each scene and how to keep the motion smooth, controlled, and visually clear. Working as a five-member team also required dividing the codebase carefully so that different modules could be handled independently. Overall, the project delivers a fully hand-built 2D disaster simulation and provides a solid base for adding more scenes, stronger interactions, and additional visual effects in the future.

10. Source Code

The complete source code for this project is available on GitHub.

[Paste GitHub repository link here]

The project is split into five modules: core (shapes and the scan-conversion algorithms), city (buildings, environment, people, props and vehicles), scenes (one file per disaster), ui (intro, hud, night, evening and ending) and main.cpp, which ties everything together.